



Model Architecture

In this assignment, we are creating a Convolutional Neural Network model to classify sign language gestures from images.

Input Layer: The input shape is $28 \times 28 \times 1$, representing 28×28 grayscale images with a single channel.

Convolutional Layers: The model includes two convolutional layers:

- First Layer: 64 filters with a kernel size of 3×3 , ReLU activation, stride 1, and valid padding (no extra pixels added).
- Second Layer: 128 filters with the same configuration, extracting deeper spatial features.

Pooling Layers: MaxPooling2D is applied after each convolutional layer to downsample the feature maps, reducing spatial dimensions by half: Pool size: 2×2 .

Dropout Layers: Dropout is used after each pooling layer to prevent overfitting:

- 30% dropout after the first pooling layer.
- 40% dropout after the second pooling layer.
- A final dropout rate of 50% applied after the fully connected layer.

Flattening Layer: The 2D feature maps from the second pooling layer are flattened into a 1D vector to connect to the fully connected layer.

Dense Layers: A fully connected layer with 256 neurons and ReLU activation is used to combine features.

Output Layer: The output layer consists of `num_classes` neurons with softmax activation, producing class probabilities for multi-class classification.



Model Architecture

Workflow:

The images in the dataset were reshaped to 28x28x1 to match the CNN's input format and normalized to the range of 0 to 1 for faster convergence. The labels were one-hot encoded to represent each class as a binary vector. The dataset was then split into training and validation sets with an 80/20 ratio.

The model was trained using the Adam optimizer and categorical cross-entropy loss function. A batch size of 128 was used, and the model was trained for 50 epochs. The validation set was used to monitor the model's performance during training and prevent overfitting.

After training, the model was evaluated on the test set using metrics such as accuracy (tracks the proportion of correctly classified examples), and validation accuracy (ensures the model generalizes well to unseen data).

Relevant Information

The model was trained using the Adam optimizer and categorical cross-entropy loss function. Early stopping was implemented to prevent overfitting.

The model achieved about 96% accuracy on the test set, showing its ability to accurately classify sign language gestures. However, there might be some challenges, including the limited dataset size and the risk of overfitting.

To address these challenges and improve the model's performance, we can experiment with different learning rates, batch sizes, and optimizer configurations. In addition, we can explore deeper architectures or more complex convolutional layers. We can also use pre-trained models on large image datasets to accelerate training and improve performance. In this way we can even increase accuracy of the model more.



1. What is one-hot encoding? Why is this important and how do you implement it in keras?

One-hot encoding is a technique used to convert categorical data into a numerical format that can be understood by machine learning algorithms. It involves creating a new binary feature for each category and assigning it a value of 1 if the data point belongs to that category, otherwise 0.

It is important because most algorithms, including neural networks, require numerical inputs. Also, it provides a clear and distinguishable representation for each category. For example, if we have a dataset of fruits: apples, oranges, and bananas. We can represent these categories using one-hot encoding:

Apple: [1, 0, 0],

Orange: [0, 1, 0]

Banana: [0, 0, 1]

In Keras, one-hot encoding can be implemented using the `to_categorical()` function. for example, if labels are a list of integers, `labels = [0, 1, 2, 0]`, in which 0 for apple, 1 for orange, 2 for banana. Then we convert it to one-hot encoded format by:

```
one_hot_labels = to_categorical(labels)
```

2. What is dropout and how does it help overfitting?

Dropout is a regularization technique that is used to prevent overfitting. During each training iteration, a random subset of neurons is temporarily dropped out. This forces the network to learn more robust features and reduces its reliance on any single neuron. It is very helpful because by making the network less sensitive to specific neurons, it generalizes better to unseen data.



3. How does ReLU differ from the sigmoid activation function?

Activation functions are very important in shaping the behavior and performance of neural networks. Two commonly used activation functions are ReLU and Sigmoid.

ReLU is a non-linear activation function that returns the input if it is positive, and 0 otherwise.

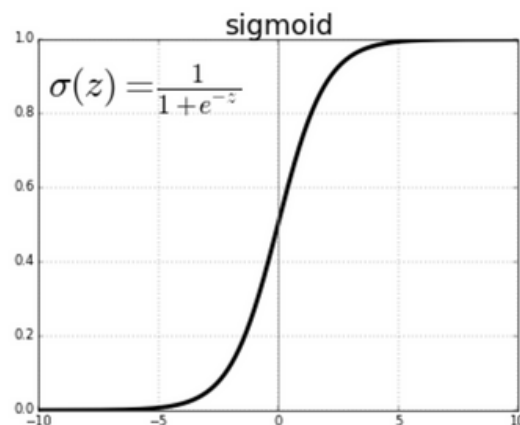
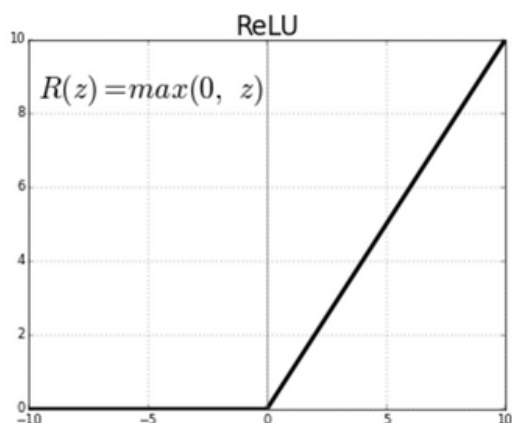
Characteristics and Applications:

- **Simplicity:** ReLU is computationally efficient and easy to implement.
- **Non-linearity:** It introduces non-linearity to the network, allowing it to learn complex patterns.
- **Avoids Vanishing Gradient:** Unlike Sigmoid, ReLU does not saturate for positive inputs, which mitigates the vanishing gradient problem and accelerates convergence during training.
- **Sparsity:** ReLU can lead to sparsity in activations, as neurons may output zero for negative inputs.
- **Commonly Used:** ReLU has become the default choice for most deep learning applications and hidden layers, especially in convolutional neural networks (CNNs).

Sigmoid is also a non-linear activation function, but it returns a value between 0 and 1. It has also slower convergence compared to ReLU.

Characteristics and Applications:

- **Output Range:** The output of the Sigmoid function always lies between 0 and 1, making it suitable for tasks where we need to model probabilities or binary classifications.
- **Smoothness:** The function is smooth and differentiable everywhere, which facilitates gradient-based optimization during training.
- **Vanishing Gradient Problem:** Sigmoid tends to saturate for very large positive or negative inputs, leading to the vanishing gradient problem, especially in deep networks.
- **Use Cases:** Historically popular in the hidden layers of neural networks, but less favored in deeper architectures due to the vanishing gradient issue.





4. Why is the softmax function necessary in the output layer?

The softmax function is used to convert the output of a neural network into a probability distribution. This is important for classification tasks because it allows us to determine the probability that a given input belongs to each class. It ensures that the output values are between 0 and 1 and sum up to 1, representing the probability of each class. In fact, it allows us to determine the most likely class for a given input. For example, if we have a 3-class classification problem, the output of the final layer might be $[2, 1, 3]$. After applying the softmax function, the output might become $[0.2, 0.1, 0.7]$, showing a 70% probability that the input belongs to the 3rd class.



5. This is a more practical calculation. Consider the following convolution network:

(a) Input image dimensions = 100x100x1

(b) Convolution layer with filters=16 and kernel size=(5,5)

(c) MaxPooling layer with pool size = (2,2)

What are the dimensions of the outputs of the convolution and max pooling layers?

Output dimensions of the convolution layer:

Using $[(W - F + 2P)/S] + 1$ formula, that is:

$[(\text{Input Height} - \text{Kernel Height} + 2\text{Padding})/\text{Stride}] + 1$, after convolution we have:

Height: $[(100 - 5 + 2*0)/1] + 1 = 95 + 1 = 96$

Width: $[(100 - 5 + 2*0)/1] + 1 = 95 + 1 = 96$

Depth: 16

So the dimensions of convolution layer would be: (96, 96, 16)

The kernel size (5,5) slides over the input image, reducing the dimensions by 4 on each side. The number of filters (16) determines the depth of the output.

Output dimensions of MaxPooling layer:

After max pooling, we have:

Height: $[(96 - 2 + 2*0)/2] + 1 = 94/2 + 1 = 47 + 1 = 48$

Width: $[(96 - 2 + 2*0)/2] + 1 = 94/2 + 1 = 47 + 1 = 48$

Depth: 16

So the dimensions of MaxPooling layer would be: (48, 48, 16)

The max pooling layer reduces the spatial dimensions by a factor of 2, while keeping the number of filters the same.